

STL (Standard Template Library) in C++

What is STL?

STL stands for **Standard Template Library**. It is a collection of **ready-made, pre-written, and highly optimized code** that comes built into C++, for the things almost every program needs to do: store a list of items, sort them, search through them, or organize data in pairs.

Without STL, a programmer would have to **write their own version** of a resizable array, a sorting algorithm, a search function, and so on, from scratch, every single time. STL means that work is **already done, tested, and optimized** - you just use it.

Example: Online Store Order List

Consider an **online store** that needs to keep a list of **customer orders**, add new orders as they come in, **sort them** by order value, and **quickly look up** a customer's order using their ID.

- A **vector** can hold the growing list of orders, since new orders keep arriving.
- The built-in **sort()** algorithm can arrange orders from highest to lowest value in one line.
- A **map** can link each customer ID directly to their order, so lookup is instant instead of searching one by one.

This is exactly what STL gives us - **ready-made containers and algorithms** for common, everyday programming needs.

Why Use STL?

- **Saves time** - no need to write your own sorting, searching, or storage logic.
- **Reliable** - STL code is heavily tested and used by millions of programs.
- **Efficient** - STL containers and algorithms are written to be fast, using well-studied techniques.

- **Consistent** - once you learn how one container works, the others follow similar patterns.

The Four Parts of STL

STL is generally divided into four parts. Each one is explained in full below - including the code you'll actually use.

1. Containers

Containers are the "**storage boxes**" that actually hold your data. Different containers store and organize data in different ways, so the right choice depends on what you need to do with it. The most common ones:

- **vector** - a **resizable array**. Elements stay in order, and you can access any element instantly by its index. The most commonly used container overall.

```
vector<int> marks;           // resizable, ordered list
```

- **map** - stores **key-value pairs** (like a dictionary). Every key is unique, and looking up a value by its key is fast.

```
map<string, int> ages;       // key -> value, unique keys
```

- **set** - stores **unique values only** (no duplicates allowed), automatically kept in sorted order.

```
set<int> uniqueIds;         // no duplicates, auto-sorted
```

- **stack** - **Last-In-First-Out (LIFO)**. Like a stack of plates - you can only add or remove from the top.

```
stack<int> undoHistory;     // last in, first out
```

- **queue** - **First-In-First-Out (FIFO)**. Like a checkout line - the first one in is the first one served.

```
queue<int> printJobs;       // first in, first out
```

- **pair** - holds exactly **two values**, which can be of different types, bundled together as one unit.

```
pair<string, int> item("Pen", 5);    // two values bundled together
```

The rest of this document gives full, runnable code for **vector** and **map** specifically, since they are the two you will reach for constantly. The others follow similar patterns once you're comfortable with these two.

vector in depth - A Resizable Array

A **vector** is like a normal array, but it can **grow or shrink automatically** as you add or remove elements. You never need to decide its size in advance. Header needed: `#include <vector>`.

- `push_back(x)` - adds `x` to the end of the vector.
- `pop_back()` - removes the last element.
- `size()` - returns how many elements are currently stored.
- `v[i]` - access the element at index `i` (like a normal array).
- `empty()` - returns `true` if the vector has no elements.
- `clear()` - removes all elements.

EXAMPLE:

```
#include <iostream>

#include <vector>

using namespace std;

int main() {

    vector<int> marks;           // starts empty

    marks.push_back(90);

    marks.push_back(85);
```

```

marks.push_back(78);

cout << "Number of marks: " << marks.size() << endl;

for (int i = 0; i < marks.size(); i++) {
    cout << marks[i] << endl;
}

return 0;
}

```

map in depth - Key-Value Pairs

A `map` stores data as **key-value pairs** - similar to a real dictionary, where you look up a **word (key)** to find its **meaning (value)**. Every key is **unique**, and looking up a value by its key is very fast. Header needed: `#include <map>`.

- `m[key] = value` - insert or update the value for a key.
- `m[key]` - access the value stored for a key.
- `m.find(key)` - returns an iterator to the key if found, or `m.end()` if not found.
- `m.erase(key)` - removes a key and its value.
- `m.size()` - returns the number of key-value pairs.

EXAMPLE:

```

#include <iostream>

#include <map>

#include <string>

using namespace std;

```

```

int main() {
    map<string, int> studentMarks;

    studentMarks["Ayesha"] = 92;
    studentMarks["Zara"] = 88;

    cout << "Ayesha's marks: " << studentMarks["Ayesha"] << endl;

    // Loop through every key-value pair
    for (auto pair : studentMarks) {
        cout << pair.first << " -> " << pair.second << endl;
    }

    return 0;
}

```

See the full runnable version in [03_map_basics.cpp](#) in the code folder.

2. Algorithms

Algorithms are **ready-made functions** that operate on a container's data, so you don't have to write your own loop-based logic for common tasks. They typically take a **range** (a start and end position) as input. Header needed: `#include <algorithm>`.

- `sort()` - arranges elements in order.

```
sort(marks.begin(), marks.end()); // 60, 78, 92
```

- `find()` - searches for a specific value.

```
auto it = find(marks.begin(), marks.end(), 78); // points to 78
```

- `reverse()` - flips the order of elements.

```
reverse(marks.begin(), marks.end());    // reverses current order
```

- `max_element()` / `min_element()` - finds the largest/smallest value.

```
auto biggest = max_element(marks.begin(), marks.end());    // *biggest = largest value
```

- `count()` - counts how many times a value appears.

```
int howMany = count(marks.begin(), marks.end(), 92);
```

EXAMPLE:

```
#include <algorithm>
```

```
vector<int> marks = {78, 92, 85};
```

```
sort(marks.begin(), marks.end());    // 78, 85, 92
```

```
reverse(marks.begin(), marks.end());    // 92, 85, 78
```

3. Iterators

An iterator **acts like a pointer** that lets you move through a container one element at a time, without needing to know how that container is actually built internally. Every container provides `begin()` (points to the first element) and `end()` (marks the position just after the last element, used as a stopping point - never a real element to read).

Iterators matter because they're the **common language** that connects containers and algorithms - `sort()`, `find()`, and every other algorithm all work by taking iterators as their input, no matter which container those iterators came from.

EXAMPLE:

```
vector<int> marks = {90, 85, 78};
```

```

for (vector<int>::iterator it = marks.begin(); it != marks.end(); it++) {
    cout << *it << endl;    // *it gives the value the iterator points to
}

// Modern, simpler alternative - the range-based for loop:
for (int m : marks) {
    cout << m << endl;
}

```

4. Functors

A functor (short for "function object") is an **object that can be called like a function**, by overloading a class's `()` operator. They are used to **customize** how an algorithm behaves - for example, telling `sort()` to sort in descending order instead of ascending.

```

class MultiplyByTwo {
public:
    int operator()(int x) {    // overloading () makes this callable
        return x * 2;
    }
};

MultiplyByTwo doubleIt;

cout << doubleIt(5) << endl;    // prints 10 - looks like a function call

```

This is a **more advanced topic**, and in modern C++ it is mostly replaced by **lambdas** (short, inline functions), so it is included here mainly so the term is recognized.

Quick Reference

- **vector:** `#include <vector>` - a resizable, ordered list.
- **map:** `#include <map>` - key-value pairs, fast lookup by key.
- **algorithm:** `#include <algorithm>` - `sort()`, `find()`, `reverse()`, and more.
- **Iterating:** use a range-based `for` loop, or `begin()/end()` with an explicit iterator.